Agentic RAG System for Macroeconomic Data

Churou Deng

April 2026

1 Introduction

As a significant breakthrough in Natural Language Processing, Large Language Models (LLMs) are increasingly used for data analysis and decision support across diverse fields and tasks. Despite their strong generative capabilities, LLMs are prone to hallucination, generating plausible yet nonfactual content (Huang et al. 2025). To address hallucinations, a general-purpose fine-tuning method known as Retrieval Augmented Generation (RAG) has emerged as a promising approach (Lewis et al. 2021). RAG combines a retriever that identifies relevant documents and a generator that creates coherent responses from this information. By combining LLMs with external knowledge bases, RAG systems can enhance response accuracy and relevance by dynamically incorporating up-to-date and verifiable information into generated outputs, compared to standalone LLMs.

While RAG has been extensively studied in general knowledge and domain-specific settings such as healthcare and scientific literature, relatively limited work has explored RAG systems grounded in economic data, particularly structured macroeconomic indicators. To address this gap, this project builds a light-weight Agentic RAG system that bridges natural-language questions and the FRED API: the system autonomously determines which series to retrieve and over which date range, fetches the data via tool calls, and generates a grounded summary answer.

2 Problem Statement and Related Work

The core research questions include: 1. Under an autonomous RAG architecture, whether a light-weight open-source LLM (LLaMA3.2) can achieve retrieval and summarization quality comparable to that of a commercial model (GPT-4o-mini). 2. Whether enhanced features such as semantic retrieval, rule-based date parsing, and self-checking loops can significantly improve retrieval accuracy and answer quality; 3. Whether fine-tuning based on GPT-generated summaries can improve the quality of LLaMA summarization.

Tool-augmented LLMs (Schick et al. 2023) demonstrate that models can invoke external tools to answer questions beyond their training cutoff. The lat-

est advancements in LLMs have enabled LLM Agents to autonomously handle tasks and reliably interact with external functionalities and APIs. The addition of tool calling, also known as function calling, has allowed complicated user requests to be possible, shining light on new use cases for Agents–such as interacting with a secure domain-specific database, and performing advanced QA tasks over domain-specific knowledge management articles (Lumer et al. 2024). Agentic RAG transforms the originally static "retrieval-generation" process into a dynamic one by introducing AI agents capable of autonomous decision-making. The agents can plan and perform multi-step reasoning based on task complexity, dynamically select retrieval strategies and invoke external tools, and continuously optimize queries and results through self-reflection. Additionally, it supports multi-agent collaboration to accomplish complex tasks. Compared to traditional RAG, Agentic RAG offers significant enhancements in flexibility, contextual understanding, and the ability to handle complex problems, making it better suited for scenarios requiring real-time information, multi-turn reasoning, and cross-domain decision-making (Singh et al. 2025). AdaptiveRAG offers a balanced strategy, seamlessly adapting between iterative and single-step retrieval-augmented LLMs, as well as non-retrieval methods, in response to a range of query complexities (Jeong et al. 2024).

In this paper, we extend the tool-augmented RAG paradigm with a multi-layer validation pipeline to improve response reliability in the macro-economic domain, where numerical precision and insight in a specific domain are critical.

The structure of this paper is as follows: Section 2 introduces the data and preprocessing. Section 3 elaborates on the principles of agentic intelligence and agentic patterns. Section 4 elaborates agentic workflow patterns. Section 5 provides details on model architecture and the fine-tuning process. Section 6 focuses on benchmarking and provides an evaluative analysis of the framework.

3 Data Collection and Preparation

3.1 Data Description

The main macroeconomic data we use is retrieved from the FRED API (Federal Reserve Bank of St. Louis). It provides hundreds of curated US macroeconomic indicators covering GDP, inflation, employment, trade, housing, monetary policy, and financial markets. For part of the series data provided by the API, please refer to Table [1].

For summary fine-tuning and retrieval accuracy testing, we use a QA dataset that contains hand-curated economic questions annotated with expected series IDs, date ranges, and (for a subset) key fact values. Question types include: 1. Single series data query or trend summarization (e.g., "Show GDP for the first quarter of 2024."); 2. Multiple series data query or trend summarization (e.g., "Compare the unemployment rate and inflation rate for 2023."); 3. Questions not related to FRED or the query tool (e.g., "What does GDP stand for?"). Queries also involve fixed date ranges and relative date expressions, designed to

Table 1: Part of the series data from FRED API

Category	Example Series	Frequency
GDP & Growth	GDP, GNP, GDPC1	Quarterly
Labor Market	UNRATE, PAYEMS, CIVPART	Monthly
Inflation	CPIAUCSL, PCEPI, CPILFESL	Monthly
Monetary Policy	FEDFUNDS, DFF, DISCOUNT	Daily / Monthly
Trade	BOPGSTB, EXPGS, IMPGS	Quarterly
Housing	PERMIT, HSN1F, MORTGAGE30US	Monthly / Weekly
Financial Markets	VIXCLS, T10Y2Y, BAA10Y	Daily
Money Supply	M1SL, M2SL, M1V, M2V	Monthly / Weekly

simulate real-world user queries. Each entry in this dataset includes: question text, expected series IDs (list), expected date range (absolute or relative), tool-call-required flag, and optionally key facts for data-point-level evaluation.

3.2 Preprocessing Steps

3.2.1 Time Series Data Analysis

We create a pipeline to compute common time series data metrics, and apply trend analysis on the data returned by the FRED API. The pipeline will then return analysed results to LLM in JSON format. This approach aims to save tokens and reduce hallucination in large language models by providing fact-based numerical analysis.

3.2.2 Series Index Guide Generation

Compact Text Series Index Guide

We create a list of the most commonly used series with their corresponding identities, names, update frequency, unit, and category. For each series, we write a detailed semantic description of its definition and economic impact in reality. This guide is used for baseline model system prompts.

Semantic Series Index Guide

Based on the compact text guide, we generate FAISS indices with metadata by encoding the series descriptions into dense embeddings using a sentence-transformer model. These embeddings would be used for dynamic series lookup to format results into LLM-readable context, which theoretically improves LLMs' ability to identify the target data associated with user queries, and thus improves their retrieval accuracy.

3.2.3 Pseudo-reference Generation

For llama3.2 fine-tuning, we reformat raw API results from GPT-4o-mini on the test questions into (question, retrieved data, reference summary) triples as a reference annotator for fine-tuning.

4 Methodology

4.1 Models

We use two baseline models: LLaMA 3.2 and GPT-4o-mini.

Table 2: An Overview of Baseline Models

Model	**Rationale**
LLaMA 3.2 (3B)	Locally deployable via Docker; no API cost; fine-tunable; representative small open-source LLM
GPT-4o-mini	Strong instruction-following and tool-use; serves as pseudo-reference for fine-tuning data and BERTScore evaluation

We develop six agents, progressively adding capabilities, to demonstrate how system design affects the performance of the RAG system. We systematically compare the two baseline models across three progressively enhanced agent variants: a baseline using a compact text indicator guide, a semantic retrieval version that dynamically selects the most relevant series via FAISS embeddings, and a final version incorporating a rule-based date parser and a three-layer self-check pipeline (relevance gating, parameter validation, and answer completeness verification). Table 3 gives an overview of the differences between agents. Retrieval accuracy is evaluated across all six variants on a curated question benchmark covering single- and multi-series queries, relative time expressions, and out-of-scope questions. See section 6 for more details.

Table 3: Agents with progressively increasing capabilities added

ID	Model	Series Retrieval	Key Enhancements
M1	LLaMA 3.2	Full text guide	Baseline
M2	LLaMA 3.2	FAISS semantic top-k	Dynamic context reduction
M3	LLaMA 3.2	FAISS semantic top-k	Check A/B/C + date parser
M4	GPT-4o-mini	Full text guide	Baseline
M5	GPT-4o-mini	FAISS semantic top-k	Dynamic context reduction
M6	GPT-4o-mini	FAISS semantic top-k	Check A/B/C + date parser

4.2 System Architecture Overview

The system follows a three-stage agentic pipeline: (1) Tool Call Extraction: Given a user question, the LLM determines which FRED series to query and over what date range; (2) Tool Call Execution: The system calls the FRED API with the extracted parameters and retrieves time-series data; (3) Summary Generation: The LLM receives the retrieved data and generates a natural-language

answer. Between these steps, we design a three-Layer self-check pipeline: (A) Relevance Gate: If the maximum cosine similarity between the question and any FRED series is below 0.3, the system returns an early rejection, preventing hallucinated series IDs for out-of-scope questions; (B) Parameter Validation: Each extracted series id is validated against the known FRED series list. Invalid IDs trigger a targeted correction re-prompt; (C) Answer Completeness: After summary generation, a separate LLM call evaluates whether the answer covers all retrieved series and addresses the original question. Incomplete answers are regenerated with targeted prompts. Figure 1 is an overview of the entire system workflow.

4.3 Key Methodological Contributions

4.3.1 Semantic Retrieval vs. Full Text Guide

The baseline appends all series descriptions to every system prompt, wasting context tokens and introducing noise. The semantic retriever uses cosine similarity in embedding space to select only the top-k most relevant series for a given question, reducing irrelevant options and improving series selection precision.

4.3.2 Rule-Based Date Parser

LLMs frequently produce date errors: future end dates, same start/end dates. A deterministic rule-based parser handles the following unambiguous patterns before the LLM is invoked:

1. Quarter expressions: "Q1 2024", "Q3 and Q4 2022" (multi-quarter merge to continuous range)
2. Half-year: "first half of 2022", "second half of 2020"
3. Relative ranges with English number words: "past two years", "last 18 months", "past decade"
4. "Since YEAR" and "from YEAR to YEAR" range patterns
5. Single unambiguous year: "in 2023", "during 2019"

Ambiguous expressions (e.g., "recently", "lately", "before Covid") are explicitly left unresolved for the LLM, following a conservative design principle that only returns a date range when fully certain.

4.3.3 Fine-tuned Summary vs. Baseline

LLaMA 3.2 is further fine-tuned on the GPT-generated reference summaries to close the summarization quality gap. See Section 5 for more details.

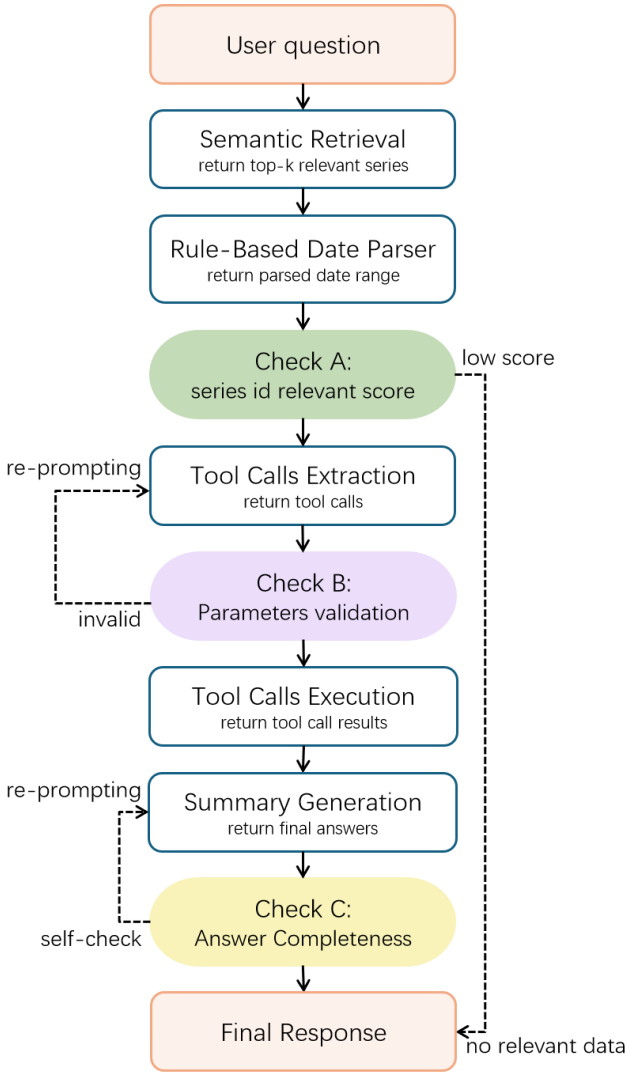

Figure 1: An Overview of System Workflow

4.4 Why Compare Multiple Models and Variants

- LLaMA 3.2 vs GPT-4o-mini: Assess whether a locally-deployable open-source model can close the performance gap with a commercial API for cost, privacy, and latency considerations.
- Baseline vs Semantic Retriever: Quantify retrieval accuracy improvement from dynamic series selection vs. full-context enumeration.
- Semantic Retriever vs Final: Isolate the contribution of the date parser and self-check pipeline to retrieval accuracy and summary quality.
- Fine-tuned LLaMA vs Base: Determine whether supervised fine-tuning on GPT-generated summaries improves LLaMA summarization quality.

5 Model Development and Fine-tuning

5.1 Evaluation Metrics

Table 4: Metrics for Model Evaluation

Metric	**Scope**	**Description**
Series id F1 Score	Retrieval	F1 score
Date Range Score	Retrieval	Continuous score (0-1) based on start/end date proximity
Overall Retrieval Score	Retrieval	$0.5 \times$ Series F1 $+ 0.5 \times$ Date Range Score
BERTScore	Summary	Semantic similarity between LLaMA and GPT summaries
Faithfulness Score	Summary	Extracted numeric claims vs. `metrics_compute()` output from FRED API
Key Fact Coverage	Summary	Whether annotated key data points appear in the answer (data-extraction questions only)
Trend Direction Accuracy	Summary	Whether described trend direction matches actual time-series slope
Hallucination Rate	Summary	Fraction of mentioned series/values absent from actual tool call results

5.2 Fine-Tuning Configuration

- Base model: LLaMA 3.2 3B
- Fine-tuning method: LoRA (Low-Rank Adaptation) for parameter-efficient training

- Training data: GPT-4o-mini generated (question, FRED data, reference summary) triples
- Objective: Supervised next-token prediction on reference summaries conditioned on question + retrieved data

5.3 Hyperparameter Configuration

Table 5: An Overview of Hyperparameter Settings

Component	**Parameter**	**Value**
Semantic Retriever	`top_k` (series returned)	k=5; tested 3–10
Check A	`min_similarity` threshold	0.3
Date Range comparison	`tolerance_days`	30 days

6 Evaluation and Comparison

6.1 Retrieval Accuracy Evaluation

Table 6: Retrieval Evaluation Results

Agent	**Series ID Score**	**Date Range Score**	**Overall Score**
M1	0.75	0.82	0.79
M2	0.91	0.84	0.87
M3	0.94	0.94	0.94
M4	0.97	0.99	0.98
M5	0.99	0.98	0.98
M6	0.97	0.99	0.98

6.2 Summary Quality Evaluation

7 Reference

References

Arslan, Muhammad, Saba Munawar, and Christophe Cruz (2024). "Business insights using RAG–LLMs: a review and case study". In: *Journal of Decision Systems* 0.0, pp. 1–30. DOI: 10.1080/12460125.2024.2410040. eprint: https://doi.org/10.1080/12460125.2024.2410040. URL: https://doi.org/10.1080/12460125.2024.2410040.

Balaguer, Angels et al. (2024). "RAG vs fine-tuning: pipelines, tradeoffs, and a case study on agriculture". In: *arXiv preprint arXiv:2401.08406*.

Barry, Mariam et al. (Jan. 2025). "GraphRAG: Leveraging Graph-Based Efficiency to Minimize Hallucinations in LLM-Driven RAG for Finance Data". In: *31st International conference on Computational Linguistics Workshop Knowledge Graph & GenAI*. Abu Dhabi, United Arab Emirates. URL: https://hal.science/hal-04907346.

Douze, Matthijs et al. (2024). "The Faiss library". In: arXiv: 2401.08281 [cs.LG].

Fabbri, Alexander R. et al. (Apr. 2021). "SummEval: Re-evaluating Summarization Evaluation". In: *Transactions of the Association for Computational Linguistics* 9, pp. 391–409. ISSN: 2307-387X. DOI: 10.1162/tacl_a_00373. eprint: https://direct.mit.edu/tacl/article-pdf/doi/10.1162/tacl_a_00373/1923949/tacl_a_00373.pdf. URL: https://doi.org/10.1162/tacl_a_00373.

Huang, Lei et al. (Jan. 2025). "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions". In: *ACM Transactions on Information Systems* 43.2, pp. 1–55. ISSN: 1558-2868. DOI: 10.1145/3703155. URL: http://dx.doi.org/10.1145/3703155.

Jeong, Soyeong et al. (June 2024). "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity". In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Ed. by Kevin Duh, Helena Gomez, and Steven Bethard. Mexico City, Mexico: Association for Computational Linguistics, pp. 7036–7050. DOI: 10.18653/v1/2024.naacl-long.389. URL: https://aclanthology.org/2024.naacl-long.389/.

Lewis, Patrick et al. (2021). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv: 2005.11401 [cs.CL]. URL: https://arxiv.org/abs/2005.11401.

Lin, Chin-Yew (2004). "Rouge: A package for automatic evaluation of summaries". In: *Text summarization branches out*, pp. 74–81.

Lumer, Elias et al. (2024). *Toolshed: Scale Tool-Equipped Agents with Advanced RAG-Tool Fusion and Tool Knowledge Bases*. arXiv: 2410.14594 [cs.CL]. URL: https://arxiv.org/abs/2410.14594.

Ng, Jun Ping and Viktoria Abrecht (2015). "Better summarization evaluation with word embeddings for ROUGE". In: *Proceedings of the 2015 conference on empirical methods in natural language processing*, pp. 1925–1930.

Schick, Timo et al. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools". In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh et al. Vol. 36. Curran Associates, Inc., pp. 68539–68551. URL: https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf.

Singh, Aditi et al. (2025). "Agentic retrieval-augmented generation: A survey on agentic rag". In: *arXiv preprint arXiv:2501.09136*.

Wang, Jingru, Wen Ding, and Xiaotong Zhu (2025). *Financial Analysis: Intelligent Financial Data Analysis System Based on LLM-RAG*. arXiv: 2504.06279 [q-fin.ST]. URL: https://arxiv.org/abs/2504.06279.

Zhang, Tianyi et al. (2019). "Bertscore: Evaluating text generation with bert". In: *arXiv preprint arXiv:1904.09675*.

Zhang, Zhaoyan et al. (Mar. 2025). "Sparse Attention Combined with RAG Technology for Financial Data Analysis". In: *Journal of Computer Science Research* 7, pp. 1–16. DOI: 10.30564/jcsr.v7i2.8933. URL: https://hal.science/hal-05180963.